

6 Mapping value types

This chapter covers

- Mapping basic properties
- Mapping embeddable components
- Controlling mapping between Java and SQL types

After spending the previous chapter almost exclusively on entities and their class- and identity-mapping options, we'll now focus on value types in their various forms. Value types are frequently encountered in classes under development. We'll separate value types into two categories: basic value-typed classes that come with the JDK, such as `String`, `Date`, primitives, and their wrappers; and developer-defined value-type classes, such as `Address` and `MonetaryAmount` in `CaveatEmptor`.

In this chapter, we'll first map persistent properties with JDK types and discuss the basic mapping annotations. We'll look at how you can work with various aspects of properties: overriding defaults, customizing access, and generated values. We'll also see how SQL is used with derived properties and transformed column values. We'll work with basic properties, temporal properties, and mapping enumerations.

We'll then examine custom value-type classes and map them as embeddable components. We'll look at how classes relate to the database schema and we'll make the classes embeddable while allowing for overriding embedded attributes. We'll complete our look at embeddable components by mapping nested components. Finally, we'll analyze how you can customize the loading and storing of property values at a lower level with flexible JPA converters, which are standardized extension points of every JPA provider.

Major new features in JPA 2

JPA 2.2 supports the Java 8 Date and Time API. There is no longer any need to use additional mapping annotations, such as `@Temporal`, that formerly were needed to annotate fields of type `java.util.Date`.

6.1 Mapping basic properties

Mapping is at the heart of the ORM technique. It makes the connection between the object-oriented world and the relational world. When we

map a persistent class, whether it's an entity or an embeddable type (more about these in section 6.2), all of its properties are considered persistent by default.

These are the default JPA rules for properties of persistent classes:

- If the property is a primitive or a primitive wrapper, or of type `String`, `BigInteger`, `BigDecimal`, `java.time.LocalDateTime`, `java.time.LocalDate`, `java.time.Instant`, `java.util.Date`, `java.util.Calendar`, `java.sql.Date`, `java.sql.Time`, `java.sql.Timestamp`, `byte[]`, `Byte[]`, `char[]`, or `Character[]`, it's automatically persistent. Hibernate or Spring Data JPA using Hibernate loads and stores the value of the property in a column with an appropriate SQL type and the same name as the property.
- Otherwise, if we annotate the class of the property as `@Embeddable`, or we map the property itself as `@Embedded`, the property is mapped as an embedded component of the owning class. We'll analyze the embedding of components later in this chapter, when we look at the `Address` and `MonetaryAmount` embeddable classes of `CaveatEmptor`.
- Otherwise, if the type of the property is `java.io.Serializable`, its value is stored in its serialized form. This may raise compatibility problems (we might have stored the information using one class format and would like to retrieve it later using another class format) and performance problems (the serialization/ deserialization operations are costly). We should always map Java classes instead of storing a series of bytes in the database. Maintaining a database with this binary information when the application may be gone in a few years will mean that the classes that the serialized version maps to are no longer available.
- Otherwise, an exception will be thrown on startup, complaining that the type of the property isn't understood.

This *configuration by exception* approach means we don't have to annotate a property to make it persistent; we only have to configure the mapping in exceptional cases. Several annotations are available in JPA to customize and control basic property mappings.

6.1.1 Overriding basic property defaults

We might not want all the properties of an entity class to be persistent. So which information ought to be persisted and which not? For example, although it makes sense to have a persistent `Item#initialPrice` property, an `Item#totalPriceIncludingTax` property shouldn't be persisted in the database if we only compute and use its value at runtime. To exclude a property, mark the field or the getter method of the property with

the annotation `@javax.persistence.Transient` or use the Java `transient` keyword. The `transient` keyword excludes fields both for Java serialization and for persistence, as it is also recognized by JPA providers. The `@javax.persistence.Transient` annotation will only exclude the field from being persisted.

To decide whether a property should be persistent or not, ask yourself these questions: Is this a basic attribute that shapes the instance? Do we need it from the very beginning, or will we calculate it based on some other properties? Does it make sense to rebuild the information after some time, or will the information no longer be significant? Is it sensitive information that we would rather avoid persisting so it can't later be revealed (such as a password in clear)? Is it information that does not have significance in some other environment (such as a local IP address that is meaningless in another network)?

We'll come back to the placement of the annotation on fields or getter methods in a moment. First, let's assume, as we have before, that Hibernate or Spring Data JPA using Hibernate will access fields directly because `@Id` has been placed on these fields. Therefore, all other JPA and Hibernate mapping annotations are also on fields.

NOTE To execute the examples from the source code, you'll first need to run the `Ch06.sql` script. The source code can be found in the `mapping-value-types` folder.

In our `CaveatEmptor` application, our goal is to not only take care of the persistence logic from the program, but also to build flexible and easy-to-change code. If we do not want to rely on property mapping defaults, we can apply the `@Basic` annotation to a particular property, such as the `initialPrice` of an `Item`:

```
@Basic(optional = false)
BigDecimal initialPrice;
```

This annotation doesn't provide many alternatives. It only has two parameters: `optional` and `fetch`. We'll talk the `fetch` option when we explore optimization strategies in section 12.1. The option shown here, `optional`, marks the property as not optional at the Java object level.

By default, all persistent properties are nullable and optional, which would mean an `Item` could have an unknown `initialPrice`. Mapping the `initialPrice` property as non-optional makes sense if you want a `NOT NULL` constraint on the `INITIALPRICE` column in the SQL schema. The generated SQL schema will automatically include a `NOT NULL` constraint for non-optional properties.

Now if the application attempts to store an `Item` without setting a value on the `initialPrice` field, an exception will be thrown before an SQL statement is sent to the database. A value is required for `initialPrice` in order to perform an `INSERT` or `UPDATE`. If we do not mark the `initialPrice` property as optional and try to save a `NULL`, the database will reject the SQL statement, and a constraint-violation exception will be thrown.

Instead of `@Basic`, we can use the `@Column` annotation to declare nullability:

```
@Column(nullable = false)
BigDecimal initialPrice;
```

We’ve now demonstrated three ways to declare whether a property value is required: with the `@Basic` annotation, with the `@Column` annotation, and earlier with the Bean Validation `@NotNull` annotation (in section 3.3.2). All have the same effect on the JPA provider: a `null` check is performed when saving, and a `NOT NULL` constraint is generated in the database schema. We recommend using the Bean Validation `@NotNull` annotation so that you can manually validate an `Item` instance and have your user interface code in the presentation layer execute validation checks automatically. There isn’t much difference in the end result, but not hitting the database with a statement that fails is cleaner.

The `@Column` annotation can also override the mapping of the property name to the database column:

```
@Column(name = "START_PRICE", nullable = false)
BigDecimal initialPrice;
```

The `@Column` annotation has a few other parameters, most of which control SQL-level details such as `catalog` and `schema` names. They’re rarely needed, and we only demonstrate them in this book when necessary.

Property annotations aren’t always on fields, and we may not want the JPA provider to access fields directly. Let’s look at customizing property access.

6.1.2 Customizing property access

The persistence engine accesses the properties of a class either directly through fields or indirectly through getter and setter methods. We will try now to answer the question, “how should we access each persistent property?” An annotated entity inherits the default from the position of the

mandatory `@Id` annotation. For example, if we declare `@Id` on a field, rather than using a getter method, all other mapping annotations for that entity are expected to be fields. Annotations are not supported on the setter methods.

The default access strategy isn't only applicable to a single entity class. Any `@Embedded` class inherits the default or explicitly declared access strategy of its owning root entity class. We'll cover embedded components later in this chapter. Furthermore, any `@MappedSuperclass` properties are accessed with the default or explicitly declared access strategy of the mapped entity class. Inheritance is the topic of chapter 7.

The JPA specification offers the `@Access` annotation for overriding the default behavior, using the parameters `AccessType.FIELD` (access through fields) and `AccessType.PROPERTY` (access through getters). When you set `@Access` on the class or entity level, all properties of the class will be accessed according to the selected strategy. Any other mapping annotations, including the `@Id`, can be set on either fields or getter methods.

We can also use the `@Access` annotation to override the access strategy of individual properties, as in the following example. Note that the position of other mapping annotations, like `@Column`, doesn't change—only how instances are accessed at runtime.

Listing 6.1 Overriding access strategy for the `name` property

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java
```

```
@Entity
public class Item {

    @Id
    @GeneratedValue(generator = "ID_GENERATOR")
    private Long id;

    @Access(AccessType.PROPERTY)
    @Column(name = "ITEM_NAME")
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name =
            !name.startsWith("AUCTION: ") ? "AUCTION: " + name : na
    }
}
```

```
}
```

- Ⓐ The `Item` entity defaults to field access. The `@Id` is on the field.
- Ⓑ The `@Access(AccessType.PROPERTY)` setting on the `name` field switches this particular property to be accessed at runtime by the JPA provider through getter/setter.
- Ⓒ Hibernate or Spring Data JPA using Hibernate calls `getName()` and `setName()` when loading and storing items.

Now turn it around: if the default (or explicit) access type of the entity were through property getter and setter methods, `@Access(AccessType.FIELD)` on a getter method would tell Hibernate or Spring Data JPA using Hibernate to access the field directly. All other mapping information would still have to be on the getter method, not the field.

Some properties don't map to a column. In particular, a derived property (like a calculated field) takes its value from an SQL expression.

6.1.3 Using derived properties

We have now come to derived properties—properties resulting from some other properties. The value of a derived property is calculated at runtime by evaluating an SQL expression declared with the `@org.hibernate.annotations.Formula` annotation, as shown in the next listing.

Listing 6.2 Two read-only derived properties

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java
```

```
@Formula(
    "CONCAT(SUBSTR(DESCRIPTION, 1, 12), '...')"
)
private String shortDescription;

@Formula(
    "(SELECT AVG(B.AMOUNT) FROM BID B WHERE B.ITEM_ID = ID)"
)
private BigDecimal averageBidAmount;
```

The SQL formulas are evaluated every time the `Item` entity is retrieved from the database and not at any other time, so the result can become outdated if other properties are modified. The properties never appear in an SQL `INSERT` or `UPDATE`, only in `SELECT`s. Evaluation occurs in the

database; the SQL formula is embedded in the `SELECT` clause when loading the instance.

The SQL formulas may refer to columns of the database table, they can call specific database SQL functions, and they can even include SQL subselects. In the previous example, the `SUBSTR()` and `CONCAT()` functions are called.

The SQL expression is passed to the underlying database as-is. Relying on vendor-specific operators or keywords may bind the mapping metadata to a particular database product. For example, the `CONCAT()` function in the preceding listing is specific to MySQL, so you should be aware that portability may be affected. Notice that unqualified column names refer to columns of the table of the class to which the derived property belongs.

Hibernate also supports a variation of formulas called *column transformers*, which allow you to write a custom SQL expression for reading *and* writing a property value. Let's investigate this capability.

6.1.4 Transforming column values

Let's now deal with information that has different representations in the object-oriented system and the relational system. Suppose a database has a column called `IMPERIALWEIGHT`, storing the weight of an `Item` in pounds. The application, however, has the property

`Item#metricWeight` in kilograms, so we would have to convert the value of the database column when reading a row from *and* writing it to the `ITEM` table. We can implement this with a Hibernate extension: the `@org.hibernate.annotations.ColumnTransformer` annotation.

Listing 6.3 Transforming column values with SQL expressions

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java

@Column(name = "IMPERIALWEIGHT")
@ColumnTransformer(
    read = "IMPERIALWEIGHT / 2.20462",
    write = "? * 2.20462"
)
private double metricWeight;
```

When reading a row from the `ITEM` table, Hibernate or Spring Data JPA using Hibernate embeds the expression `IMPERIALWEIGHT / 2.20462`, so the calculation occurs in the database, and the metric value is returned in the result to the application layer. For writing to the column, Hibernate or Spring Data JPA using Hibernate sets the metric value on the mandato-

ry, single placeholder (the question mark), and the SQL expression calculates the actual value to be inserted or updated.

Hibernate also applies column converters in query restrictions. For example, the query in the following listing retrieves all items with a weight of 2 kilograms.

Listing 6.4 Applying column converters in query restrictions

```
Path: Ch06/mapping-value-types/src/test/java/com/manning/javapersistenc
➡ /ch06/MappingValuesJPATest.java
```

```
List<Item> result =
    em.createQuery("SELECT i FROM Item i WHERE i.metricWeight = :w")
        .setParameter("w", 2.0)
        .getResultList();
```

The actual SQL executed for this query contains the following restriction in the `WHERE` clause:

```
// . . .
where
    i.IMPERIALWEIGHT / 2.20462=?
```

Note that the database will probably not be able to rely on an index for this restriction; a full table scan would be performed because the weights for *all* `ITEM` rows have to be calculated to evaluate the restriction.

Another special kind of property relies on database-generated values.

6.1.5 Generated and default property values

The database sometimes generates a property value, and this usually happens when we insert a row for the first time. Examples of database-generated values are creation timestamps, a default price for an item, or a trigger that runs for every modification.

Typically, Hibernate (or Spring Data JPA using Hibernate) applications need to refresh instances that contain properties for which the database generates values after saving. This means the application would have to make another round trip to the database to read the value after inserting or updating a row. Marking properties as generated, however, lets the application delegate this responsibility to Hibernate or Spring Data JPA using Hibernate. Essentially, whenever an SQL `INSERT` or `UPDATE` is issued for an entity that has declared generated properties, the SQL does a `SELECT` immediately afterward to retrieve the generated values.

We use the `@org.hibernate.annotations.Generated` annotation to mark generated properties. For temporal properties, we use the `@CreationTimestamp` and `@UpdateTimestamp` annotations. The `@CreationTimestamp` annotation is used to mark the `createdOn` property. This tells Hibernate or Spring Data using Hibernate to generate the property value automatically. In this case, the value is set to the current date before the entity instance is inserted into the database. The other similar built-in annotation is `@UpdateTimestamp`, which generates the property value automatically when an entity instance is updated.

Listing 6.5 Database-generated property values

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java

@CreationTimestamp
private LocalDate createdOn;

@UpdateTimestamp
private LocalDateTime lastModified;

@Column(insertable = false)
@ColumnDefault("1.00")
@Generated(
    org.hibernate.annotations.GenerationTime.INSERT
)
private BigDecimal initialPrice;
```

Available settings for the `GenerationTime` enum are `ALWAYS` and `INSERT`. With `GenerationTime.ALWAYS`, Hibernate or Spring Data JPA using Hibernate refreshes the entity instance after every SQL `UPDATE` or `INSERT`. With `GenerationTime.INSERT`, the refreshing only occurs after an SQL `INSERT` to retrieve the default value provided by the database. We can also map the `initialPrice` property as not `insertable`. The `@ColumnDefault` annotation sets the default value of the column when Hibernate or Spring Data JPA using Hibernate exports and generates the SQL schema DDL.

Timestamps are frequently automatically generated either by the database, as in the previous example, or by the application. As long as we are using JPA 2.2 and the Java 8 `LocalDate`, `LocalDateTime`, and `LocalTime` classes, we don't need to use the `@Temporal` annotation. The enumerated Java 8 classes from the `java.time` package include the temporal precision by themselves: the date, the date and the time, or only the time. Let's take a look at the uses of the `@Temporal` annotation that you may still encounter.

6.1.6 The @Temporal annotation

The JPA specification allows you to annotate temporal properties with `@Temporal` to declare the accuracy of the SQL data type of the mapped column. The Java temporal types before Java 8 are `java.util.Date`, `java.util.Calendar`, `java.sql.Date`, `java.sql.Time`, and `java.sql.Timestamp`. The following listing provides an example of using the `@Temporal` annotation.

Listing 6.6 Property of a temporal type that must be annotated with `@Temporal`

```
@CreationTimestamp
@Temporal(TemporalType.DATE)
private Date createdOn;

@UpdateTimestamp
@Temporal(TemporalType.TIMESTAMP)
private Date lastModified;
```

Available `TemporalType` options are `DATE`, `TIME`, and `TIMESTAMP`, establishing which part of the temporal value should be stored in the database. The default is `TemporalType.TIMESTAMP` when no `@Temporal` annotation is present.

Another special property type is represented by the enumerations.

6.1.7 Mapping enumerations

An *enumeration type* is a common Java idiom where a class has a constant (small) number of immutable instances. In `CaveatEmptor`, for example, we can apply this to auctions having a limited number of types:

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/AuctionType.java
```

```
public enum AuctionType {
    HIGHEST_BID,
    LOWEST_BID,
    FIXED_PRICE
}
```

We can now set the appropriate `auctionType` on each `Item`:

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java
```

```
@NotNull
```

```
@Enumerated(EnumType.STRING)
private AuctionType auctionType = AuctionType.HIGHEST_BID;
```

Without the `@Enumerated` annotation, Hibernate or Spring Data JPA using Hibernate would store the `ORDINAL` position of the value. That is, it would store 1 for `HIGHEST_BID`, 2 for `LOWEST_BID`, and 3 for `FIXED_PRICE`. This is a brittle default; if you make changes to the `AuctionType` enum and add a new instance, existing values may no longer map to the same position and break the application. The `EnumType.STRING` option is, therefore, a better choice; Hibernate or Spring Data JPA using Hibernate can store the label of the enum value as is.

This completes our tour of basic properties and their mapping options. So far we have looked at the properties of JDK-supplied types such as `String`, `Date`, and `BigDecimal`. The domain model also has custom value-typed classes—those with a composition association in the UML diagram.

6.2 Mapping embeddable components

The mapped classes of our domain model have all been entity classes so far, each with its own lifecycle and identity. The `User` class, however, has a special kind of association with the `Address` class, as shown in figure 6.1.

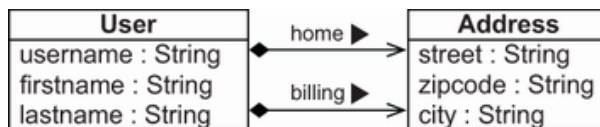


Figure 6.1 Composition of `User` and `Address`

In object-modeling terms, this association is a kind of *aggregation*—a *part-of* relationship. An aggregation is a form of association, but it has some additional semantics concerning the lifecycle of objects. In this case, we have an even stronger form, *composition*, where the lifecycle of the part is fully dependent on the lifecycle of the whole. An `Address` object cannot exist in the absence of the `User` object, so a composed class in UML, such as `Address`, is often a candidate value type for the object/relational mapping.

6.2.1 The database schema

We can map such a composition relationship with `Address` as a value type (with the same semantics as `String` or `BigDecimal`) and with `User` as an entity. The targeted SQL schema is shown in figure 6.2.

<< Table >> USERS	
ID <<PK>>	Component columns
USERNAME	
FIRSTNAME	
LASTNAME	
STREET	
ZIPCODE	
CITY	
BILLING_STREET	
BILLING_ZIPCODE	
BILLING_CITY	

Figure 6.2 The columns of the components are embedded in the entity table.

There is only one mapped table, `USERS`, for the `User` entity. This table embeds all the details of the components, where a single row holds a particular `User` and their `homeAddress` and `billingAddress`. If another entity has a reference to an `Address`—such as `Shipment#deliveryAddress`—the `SHIPMENT` table will also have all the columns needed to store an `Address`.

This schema reflects value type semantics: a particular `Address` can't be shared; it doesn't have its own identity. Its primary key is the mapped database identifier of the owning entity. An embedded component has a dependent lifecycle: when the owning entity instance is saved, the component instance is saved. When the owning entity instance is deleted, the component instance is deleted. No special SQL needs to be executed for this; all the data is in a single row.

Having “more classes than tables” is how fine-grained domain models are supported. Let's write the classes and mappings for this structure.

6.2.2 Making classes embeddable

Java has no concept of composition—a class or property can't be marked as a component. The only difference between a component and an entity is the database identifier: a component class has no individual identity, so the component class requires no identifier property or identifier mapping. It's a simple POJO, as in the following listing.

Listing 6.7 `Address` class: An embeddable component

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Address.java
```

```
@Embeddable
public class Address {
    @NotNull
    @Column(nullable = false)
    private String street;

    @NotNull
```

```

        @Column(nullable = false, length = 5)
        private String zipcode;

        @NotNull
        @Column(nullable = false)
        private String city;

        public Address() {
        }

        public Address(String street, String zipcode, String city) {
            this.street = street;
            this.zipcode = zipcode;
            this.city = city;
        }
        //getters and setters
    }

```

Ⓐ Instead of `@Entity`, this component POJO is marked with `@Embeddable`. It has no identifier property.

Ⓑ The `@NotNull` annotation is ignored by the DDL generation.

Ⓒ `@Column(nullable=false)` is used for DDL generation.

Ⓓ The length argument of the `@Column` annotation will override the default generation of a column as `VARCHAR(255)`.

Ⓔ The type of the `city` column will by default be `VARCHAR(255)`.

Ⓕ Hibernate or Spring Data JPA using Hibernate call this no-argument constructor to create an instance and then populate the fields directly.

Ⓖ We can have additional (public) constructors for convenience.

In the preceding listing the properties of the embeddable class are all persistent by default, just like the properties of a persistent entity class. The property mappings can be configured with the same annotations, such as `@Column` or `@Basic`. The properties of the `Address` class map to the columns `STREET`, `ZIPCODE`, and `CITY`, and they are constrained with `NOT NULL`. That's the entire mapping.

Problem: Hibernate Validator doesn't generate `NOT NULL` constraints

At the time of writing, an open flaw remains with Hibernate Validator: Hibernate won't map `@NotNull` constraints on embeddable component properties to `NOT NULL` constraints when generating the database schema. Hibernate will only use `@NotNull` on the components' properties at runtime for Bean Validation. We have to map the property with

`@Column(nullable = false)` to generate the constraint in the schema. The Hibernate bug database is tracking this problem as HVAL-3 (see <http://mng.bz/lR0R>).

There's nothing special about the `User` entity.

Listing 6.8 `User` class containing a reference to an `Address`

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;

    private Address homeAddress;    ❸

    // . . .
}
```

❸ The `Address` is `@Embeddable` so no annotation is needed here.

In the preceding listing, Hibernate or Spring Data using Hibernate detects that the `Address` class is annotated with `@Embeddable`; the `STREET`, `ZIPCODE`, and `CITY` columns are mapped on the `USERS` table, the owning entity's table.

When we talked about property access earlier in this chapter, we mentioned that embeddable components inherit their access strategy from their owning entity. This means Hibernate or Spring Data using Hibernate will access the properties of the `Address` class with the same strategy as for `User` properties. This inheritance also affects the placement of mapping annotations in embeddable component classes. The rules are as follows:

- If the owning `@Entity` of an embedded component is mapped with field access, either implicitly with `@Id` on a field or explicitly with `@Access(AccessType.FIELD)` on the class, all mapping annotations of the embedded component class are expected on fields of the component class. Annotations are expected on the fields of the `Address` class, and the fields are directly read and written at runtime. Getter and setter methods on `Address` are optional.

- If the owning `@Entity` of an embedded component is mapped with property access, either implicitly with `@Id` on a getter method or explicitly with `@Access(AccessType.PROPERTY)` on the class, all mapping annotations of the embedded component class are expected on getter methods of the component class. Values are read and written by calling getter and setter methods on the embeddable component class.
- If the embedded property of the owning entity class—`User#homeAddress` in listing 6.8—is marked with `@Access(AccessType.FIELD)`, annotations are expected on the fields of the `Address` class, and fields are accessed at runtime.
- If the embedded property of the owning entity class—`User#homeAddress` in listing 6.8—is marked with `@Access(AccessType.PROPERTY)`, annotations are expected on getter methods of the `Address` class, and access is made using getter and setter methods at runtime.
- If `@Access` annotates the embeddable class itself, the selected strategy will be used for reading mapping annotations on the embeddable class and runtime access.

Let's now compare field-based and property-based access. Why should you use one or the other?

- *Field-based access*—When you use field-based access, you can omit getter methods for the fields that should not be exposed. Also, fields are declared on a single line, while accessor methods are spread out on multiple lines, so field-based access will make the code more readable.
- *Property-based access*—Accessor methods may execute additional logic. If this is what you want to happen when persisting an object, you can use property-based access. If the persistence would like to avoid these additional actions, you can use field-based access.

There's one more thing to remember: there's no elegant way to represent a `null` reference to an `Address`. Consider what would happen if the columns `STREET`, `ZIPCODE`, and `CITY` were nullable. If you load a `User` without any address information, what should be returned by `someUser.getHomeAddress()`? A `null` would be returned in this case. Hibernate or Spring Data using Hibernate also stores a `null` embedded property as `NULL` values in all mapped columns of the component. Consequently, if you store a `User` with an “empty” `Address` (an `Address` instance exists, but all its properties are `null`), no `Address` instance will be returned when loading the `User`. This can be counterintuitive; you probably shouldn't have nullable columns anyway, and avoid ternary logic, as you will most probably want your user to have a real address.

We should override the `equals()` and `hashCode()` methods of `Address` to compare instances by value. However, this isn't critically im-

portant as long as we don't have to compare instances, such as by putting them in a `HashSet`. We'll discuss this problem in the context of collections in section 8.2.1.

In a realistic scenario, a user would probably have separate addresses for different purposes. Figure 6.1 showed an additional composition relationship between `User` and `Address`: the `billingAddress`.

6.2.3 Overriding embedded attributes

The `billingAddress` is another embedded component property of the `User` class that we'll need to use, so another `Address` has to be stored in the `USERS` table. This creates a mapping conflict: so far, we only have `STREET`, `ZIPCODE`, and `CITY` columns in the schema to store one `Address`.

We'll need additional columns to store another `Address` for each `USERS` row. When we map the `billingAddress`, we can override the column names.

Listing 6.9 Overriding the column names

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @Embedded                                ①
    @AttributeOverride(name = "street",      ②
        column = @Column(name = "BILLING_STREET")) ③
    @AttributeOverride(name = "zipcode",     ④
        column = @Column(name = "BILLING_ZIPCODE", length = 5)) ⑤
    @AttributeOverride(name = "city",        ⑥
        column = @Column(name = "BILLING_CITY")) ⑦
    private Address billingAddress;

    public Address getBillingAddress() {
        return billingAddress;
    }

    public void setBillingAddress(Address billingAddress) {
        this.billingAddress = billingAddress;
    }
    // . . .
}
```

① The `billingAddress` field is marked as embedded. The `@Embedded` annotation actually isn't necessary. You can mark either the component class or the property in the owning entity class (using both doesn't hurt

but it offers no advantage). The `@Embedded` annotation is useful if you want to map a third-party component class without a source and no annotations but using the right getter and setter methods (like regular JavaBeans).

⑧ The repeatable `@AttributeOverride` annotation selectively overrides property mappings of the embedded class. In this example we override all three properties and provide different column names. Now we can store two `Address` instances in the `USERS` table, with each instance in a different set of columns (look back at the schema again in figure 6.2).

Each `@AttributeOverride` annotation for a component property is “complete”; any JPA or Hibernate annotation on the overridden property is ignored. This means the `@Column` annotations on the `Address` class are ignored, so all `BILLING_*` columns are `NULL` able! (Bean Validation still recognizes the `@NotNull` annotation on the component property, though; only the persistence annotations are overridden.)

We’ll create two Spring Data JPA repository interfaces to interact with the database. The `UserRepository` interface only extends `CrudRepository`, and it will inherit all the methods from this interface. It is generified by `User` and `Long`, as it manages `User` entities having `Long` IDs.

Listing 6.10 The `UserRepository` interface

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/repositories/UserRepository.java

public interface UserRepository extends CrudRepository<User, Long> {
}
```

The `ItemRepository` interface extends `CrudRepository`, and it will inherit all the methods from this interface. Additionally, it declares the `findByMetricWeight` method, following the Spring Data JPA naming conventions. It is generified by `Item` and `Long`, as it manages `Item` entities having `Long` IDs.

Listing 6.11 The `ItemRepository` interface

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/repositories/ItemRepository.java

public interface ItemRepository extends CrudRepository<Item, Long> {
    Iterable<Item> findByMetricWeight(double weight);
}
```



```

        () -> assertEquals(AuctionType.HIGHEST_BID,
                           items.get(0).getAuctionType()),
        () -> assertEquals("description...",
                           items.get(0).getShortDescription()),
        () -> assertEquals(2.0, items.get(0).getMetricWeight()),
        () -> assertEquals(LocalDate.now(),
                           items.get(0).getCreatedOn()),
        () ->
            assertTrue(ChronoUnit.SECONDS.between(
                LocalDateTime.now(),
                items.get(0).getLastModified()) < 1),
        () -> assertEquals(new BigDecimal("1.00"),
                           items.get(0).getInitialPrice())

    };

}
}

```

- Ⓐ We extend the test using `SpringExtension`. This extension is used to integrate the Spring test context with the JUnit 5 Jupiter test.
- Ⓑ The Spring test context is configured using the beans defined in the `SpringDataConfiguration` class.
- Ⓒ A `UserRepository` bean is injected by Spring through autowiring.
- Ⓓ An `ItemRepository` bean is injected by Spring through autowiring. This is possible as the `com.manning.javapersistence.ch06.repositories` package where `UserRepository` and `ItemRepository` are located was used as the argument of the `@EnableJpaRepositories` annotation on the `SpringDataConfiguration` class. You can look back at chapter 2 for a refresher on what the `SpringDataConfiguration` class looks like.
- Ⓔ Create and set a user.
- Ⓕ Save it to the repository.
- Ⓖ Create and set an item.
- Ⓗ Save it to the repository.
- Ⓘ Get the list of all users.
- Ⓙ Get the list of items having the metric 2.0.
- Ⓚ Check the size of the list of users.
- Ⓛ Check the name.

- Ⓜ Check the street address.
- Ⓝ Check the ZIP code.
- Ⓞ Check the city of the first user in the list.
- Ⓟ Check the size of the list of items.
- Ⓠ Check the name of the first item.
- Ⓡ Check its description.
- Ⓢ Check the auction type.
- Ⓣ Check the short description.
- Ⓤ Check its metric weight.
- Ⓥ Check the creation date.
- Ⓦ Check the last modification date and time.
- Ⓧ Check the initial price of the first item in the list. The last modification date and time is checked against the current date and time, to ensure it is within 1 second (allowing for the retrieval delay).

The domain model in the preceding listing can further improve reusability and be made more fine-grained by nesting embedded components.

6.2.4 Mapping nested embedded components

Let's consider the `Address` class and how it encapsulates address details; instead of a having a simple `city` string, we can move this detail into a new `City` embeddable class. The revised domain model diagram is shown in figure 6.3. The SQL schema targeted for the mapping still has only one `USERS` table, as shown in figure 6.4. The source code that follows in listings 6.13 and 6.14 can be found in the `mapping-value-types2` folder.

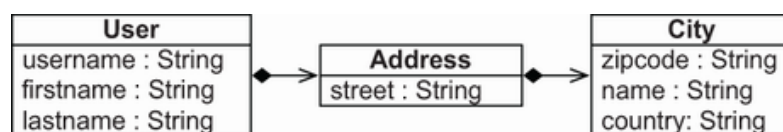


Figure 6.3 Nested composition of `Address` and `City`

<< Table >> USERS	
ID <<PK>>	
USERNAME	
FIRSTNAME	
LASTNAME	
STREET	Component columns
ZIPCODE	
CITY	
COUNTRY	

Figure 6.4 Embedded columns hold Address and City details.

An embeddable class can have an embedded property, and Address has a city property.

Listing 6.13 The Address class with a city property

Path: Ch06/mapping-value-types2/src/main/java/com/manning/javapersisten
 ➡ /ch06/model/Address.java

```
@Embeddable
public class Address {
    @NotNull
    @Column(nullable = false)
    private String street;

    @NotNull
    @AttributeOverride(
        name = "name",
        column = @Column(name = "CITY", nullable = false)
    )
    private City city;
    // . . .
}
```

We'll create the embeddable City class with only basic properties.

Listing 6.14 The embeddable City class

Path: Ch06/mapping-value-types2/src/main/java/com/manning/javapersisten
 ➡ /ch06/model/City.java

```
@Embeddable
public class City {
    @NotNull
    @Column(nullable = false, length = 5)
    private String zipcode;

    @NotNull
    @Column(nullable = false)
    private String name;
}
```

```

        @NotNull
        @Column(nullable = false)
        private String country;
        // . . .
    }

```

We could continue this kind of nesting by creating a `Country` class, for example. All embedded properties, no matter how deep they are in the composition, are mapped to columns of the owning entity's table, which here is the `USERS` table.

The `name` property of the `City` class is mapped to the `CITY` column. This can be achieved with either an `@AttributeOverride` in `Address` (as demonstrated) or an override in the root entity class, `User`. Nested properties can be referenced with dot notation; for example, on `User#address`, `@AttributeOverride(name = "city.name")` references the `Address#city#name` attribute.

We'll come back to embedded components in section 8.2, where we'll look at mapping collections of components and using references from a component to an entity.

At the beginning of this chapter, we analyzed basic properties and how Hibernate or Spring Data JPA using Hibernate can map a JDK type such as `java.lang.String` to an appropriate SQL type. Let's find out more about this type system and how values are converted at a lower level.

6.3 Mapping Java and SQL types with converters

Until now, we've assumed that Hibernate or Spring Data JPA using Hibernate will select the right SQL type when we map a `java.lang.String` property. But what is the correct mapping between the Java and SQL types, and how can we control it? We'll shape a correspondence between these types as we delve into the specifics.

6.3.1 Built-in types

Any JPA provider has to support a minimum set of Java-to-SQL type conversions. Hibernate and Spring Data JPA using Hibernate support all of these mappings, as well as some additional adapters that aren't standard but are useful in practice. First let's look at the Java primitives and their SQL equivalents.

PRIMITIVE AND NUMERIC TYPES

The built-in types shown in table 6.1 map Java primitives and their wrappers to appropriate SQL standard types. We've also included some other

numeric types. The names in the Name column are Hibernate-specific; we'll use them later when customizing type mappings.

Table 6.1 Java primitive types that map to SQL standard types

Name	Java type	ANSI SQL type
integer	int, java.lang.Integer	INTEGER
long	long, java.lang.Long	BIGINT
short	short, java.lang.Short	SMALLINT
float	float, java.lang.Float	FLOAT
double	double, java.lang.Double	DOUBLE
byte	byte, java.lang.Byte	TINYINT
boolean	boolean, java.lang.Boolean	BOOLEAN
big_decimal	java.math.BigDecimal	NUMERIC
big_integer	java.math.BigInteger	NUMERIC

You probably noticed that your DBMS product doesn't support some of the SQL types listed. These SQL type names are ANSI-standard type names. Most DBMS vendors ignore this part of the SQL standard, usually because their legacy type systems preceded the standard. However, JDBC provides a partial abstraction of vendor-specific data types, allowing Hibernate to work with ANSI-standard types when executing DML statements such as `INSERT` and `UPDATE`. For product-specific schema generation, Hibernate translates from the ANSI-standard type to an appropriate vendor-specific type using the configured SQL dialect. This means we usually don't have to worry about SQL data types if we let Hibernate create the schema for us.

If we have an existing schema or we need to know the native data type for our DBMS, we can look at the source of our configured SQL dialect. For example, the `H2Dialect` that ships with Hibernate contains this mapping from the ANSI `NUMERIC` type to the vendor-specific `DECIMAL` type: `registerColumnType(Types.NUMERIC, "decimal($p,$s)")`.

The `NUMERIC` SQL type supports decimal precision and scale settings. The default precision and scale setting for a `BigDecimal` property, for example, is `NUMERIC(19, 2)`. To override this for schema generation, apply the `@Column` annotation on the property and set its `precision` and `scale` parameters.

Next are the types that map to strings in the database.

CHARACTER TYPES

Table 6.2 shows types that map character and string value representations.

Table 6.2 Adapters for character and string values

Name	Java type	ANSI SQL type
<code>string</code>	<code>java.lang.String</code>	<code>VARCHAR</code>
<code>character</code>	<code>char[]</code> , <code>Character[]</code> , <code>java.lang.String</code>	<code>CHAR</code>
<code>yes_no</code>	<code>boolean</code> , <code>java.lang.Boolean</code>	<code>CHAR(1)</code> , 'Y' or 'N'
<code>true_false</code>	<code>boolean</code> , <code>java.lang.Boolean</code>	<code>CHAR(1)</code> , 'T' or 'F'
<code>class</code>	<code>java.lang.Class</code>	<code>VARCHAR</code>
<code>locale</code>	<code>java.util.Locale</code>	<code>VARCHAR</code>
<code>timezone</code>	<code>java.util.TimeZone</code>	<code>VARCHAR</code>
<code>currency</code>	<code>java.util.Currency</code>	<code>VARCHAR</code>

The Hibernate type system picks an SQL data type depending on the declared length of a string value: if the `String` property is annotated with `@Column(length = ...)` or `@Length` in Bean Validation, Hibernate selects the right SQL data type for the given string size. This selection also depends on the configured SQL dialect. For example, for MySQL, a length of up to 65,535 produces a regular `VARCHAR(length)` column when the schema is generated by Hibernate. For a length of up to 16,777,215, a MySQL-specific `MEDIUMTEXT` data type is produced, and even greater lengths use a `LONGTEXT`. The default length of Hibernate for all `java.lang.String` properties is 255, so without any further mapping, a `String` property maps to a `VARCHAR(255)` column. You can customize this type selection by extending the class of your SQL dialect; read the dialect documentation and source code to find out the details for your DBMS product.

A database usually enables the internationalization of text with a sensible (UTF-8) default character set for the entire database or at least whole tables. This is a DBMS-specific setting. If you need more fine-grained control and want to switch to the nationalized variants of character data types (e.g., `NVARCHAR`, `NCHAR`, or `NCLOB`), annotate the property mapping with `@org.hibernate.annotations.Nationalized`.

Also built in are some special converters for legacy databases or DBMSs with limited type systems, such as Oracle. The Oracle DBMS doesn't even have a truth-valued data type; the only data type required by the relational model. Many existing Oracle schemas, therefore, represent Boolean values with `Y/N` or `T/F` characters. Or—and this is the default in Hibernate's Oracle dialect—a column of type `NUMBER(1,0)` is expected and generated. Again, refer to the SQL dialect of the DBMS if you want to know all mappings from ANSI data type to vendor-specific type.

Next are the types that map to dates and times in the database.

DATE AND TIME TYPES

Table 6.3 lists types associated with dates, times, and timestamps.

Table 6.3 Date and time types

Name	Java type	ANSI SQL type
date	java.util.Date, java.sql.Date	DATE
time	java.util.Date, java.sql.Time	TIME
timestamp	java.util.Date, java.sql.Time- stamp	TIMESTAMP
calendar	java.util.Calen- dar	TIMESTAMP
calendar_date	java.util.Calen- dar	DATE
duration	java.time.Dura- tion	BIGINT
instant	java.time.In- stant	TIMESTAMP
localdatetime	java.time.Local- DateTime	TIMESTAMP
localdate	java.time.Local- Date	DATE
localtime	java.time.Local- Time	TIME
offsetdatetime	java.time.Off- setDateTime	TIMESTAMP
offsettime	java.time.Off- setTime	TIME
zoneddatetime	java.time.Zoned- DateTime	TIMESTAMP

In the domain model, we can represent date and time data as either `java.util.Date`, `java.util.Calendar`, or the subclasses of `java.u-`

`util.Date` defined in the `java.sql` package, or the Java 8 classes from the `java.time` package. The best decision at this time is to use the Java 8 API in the `java.time` package. These classes may represent a date, a time, a date with a time, or they may even include the offset to the UTC zone (`OffsetDateTime` and `OffsetTime`). JPA 2.2 officially supports the Java 8 date and time classes.

Hibernate's behavior for `java.util.Date` properties might be a surprise at first: when storing a `java.util.Date`, Hibernate won't return a `java.util.Date` after loading. It will return a `java.sql.Date`, a `java.sql.Time`, or a `java.sql.Timestamp`, depending on whether the property is mapped with `TemporalType.DATE`, `TemporalType.TIME`, or `TemporalType.TIMESTAMP`.

Hibernate has to use the JDBC subclass when loading data from the database because the database types have higher accuracy than `java.util.Date`. A `java.util.Date` has millisecond accuracy, but a `java.sql.Timestamp` includes nanosecond information that may be present in the database. Hibernate won't cut off this information to fit the value into `java.util.Date`, which may lead to problems when trying to compare `java.util.Date` values with the `equals()` method; it's not symmetric with the `java.sql.Timestamp` subclass's `equals()` method.

The solution in such a case is simple and not even specific to Hibernate: don't call `aDate.equals(bDate)`. You should always compare dates and times by comparing Unix time milliseconds (assuming you don't care about the nanoseconds): `aDate.getTime() > bDate.getTime()`, for example, is `true` if `aDate` is a later time than `bDate`. But be careful: collections such as `HashSet` call the `equals()` method as well. Don't mix `java.util.Date` and `java.sql.Date|Time|Timestamp` values in such a collection.

You won't have this kind of problem with a `Calendar` property. When storing a `Calendar` value, Hibernate will always return a `Calendar` value, created with `Calendar.getInstance()`—the actual type depends on locale and time zone.

Alternatively, you can write your own *converter*, as shown in section 6.3.2, and transform any instance of a `java.sql` temporal type from Hibernate into a plain `java.util.Date` instance. A custom converter is also a good starting point if, for example, a `Calendar` instance should have a non-default time zone after the value is loaded from the database.

All these concerns will go away if you choose to represent date and time data using the Java 8 classes `LocalDate`, `LocalTime`, `LocalDateTime`, as previously demonstrated in section 6.1.5. As you may still encounter a

lot of code that uses the old classes, you should be aware of the problems that these can raise.

Next are types that map to binary data and large values in the database.

BINARY AND LARGE VALUE TYPES

Table 6.4 lists types for handling binary data and large values. Note that only `binary` is supported as the type of an identifier property.

If a property in the persistent Java class is of type `byte[]`, Hibernate maps it to a `VARBINARY` column. The real SQL data type will depend on the dialect; for example, in PostgreSQL, the data type is `BYTEA`, and in Oracle DBMS it's `RAW`. In some dialects, the `length` set with `@Column` also affects the selected native type; for example, `LONG RAW` is used for a length of 2,000 and greater in Oracle. In MySQL, the default SQL data type will be `TINYBLOB`. Depending on the `length` set with `@Column`, it may be `BLOB`, `MEDIUMBLOB`, or `LONGBLOB`.

Table 6.4 Binary and large value types

Name	Java type	ANSI SQL type
<code>binary</code>	<code>byte[]</code> , <code>java.lang.Byte[]</code>	<code>VARBINARY</code>
<code>text</code>	<code>java.lang.String</code>	<code>CLOB</code>
<code>clob</code>	<code>java.sql.Clob</code>	<code>CLOB</code>
<code>blob</code>	<code>java.sql.Blob</code>	<code>BLOB</code>
<code>serializable</code>	<code>java.io.Serializable</code>	<code>VARBINARY</code>

A `java.lang.String` property is mapped to an SQL `VARCHAR` column and the same is true for `char[]` and `Character[]`. As we've discussed, some dialects register different native types depending on the declared length.

Hibernate initializes the property value right away when the entity instance that holds the property variable is loaded. This is inconvenient when you have to deal with potentially large values, so you'll usually want to override this default mapping. The JPA specification has a convenient shortcut annotation for this purpose, `@Lob`:

```

@Entity
public class Item {
    @Lob
    private byte[] image;
    @Lob
    private String description;
}

```

This maps the `byte[]` to an SQL BLOB data type and the `String` to a CLOB. Unfortunately, you still won't get lazy loading with this design. Hibernate or Spring Data JPA using Hibernate would have to intercept field access and, for example, load the bytes of the `image` when you call `someItem.getImage()`. This approach requires bytecode instrumentation of the classes after compilation, for the injection of extra code. We'll discuss lazy loading through bytecode instrumentation and interception in section 12.1.2.

Alternatively, you can switch the type of property in the Java class. JDBC supports large objects (LOBs) directly. If the Java property is `java.sql.Clob` or `java.sql.Blob`, you'll get lazy loading without bytecode instrumentation:

```

@Entity
public class Item {
    @Lob
    private java.sql.Blob imageBlob;
    @Lob
    private java.sql.Clob description;
}

```

What does BLOB/CLOB mean?

Jim Starkey, who came up with the idea of LOBs, says that the marketing department created the terms BLOB and CLOB. BLOB is interpreted as Binary Large Object: binary data (usually a multimedia object—image, video, or audio) stored as a single entity. CLOB means Character Large Object—character data stored in a separate location that the table only references.

These JDBC classes include behavior for loading values on demand. When the owning entity instance is loaded, the property value is a placeholder, and the real value isn't immediately materialized. Once you access the property, within the same transaction, the value is materialized or even streamed directly (to the client) without consuming temporary memory:

```

Item item = em.find(Item.class, ITEM_ID);
InputStream imageDataStream = item.getImageBlob().getBinaryStream();
ByteArrayOutputStream outputStream = new ByteArrayOutputStream();
StreamUtils.copy(imageDataStream, outputStream);
byte[] imageBytes = outputStream.toByteArray();

```

Ⓐ Stream the bytes directly.

Ⓑ Or materialize them into memory.

Ⓒ `org.springframework.util.StreamUtils` is a class providing utility methods for dealing with streams.

The downside is that the domain model is then bound to JDBC; in unit tests you can't access LOB properties without a database connection.

To create and set a `Blob` or `Clob` value, Hibernate offers some convenience methods. This example reads `byteLength` bytes from an `InputStream` directly into the database, without consuming temporary memory:

```

Session session = em.unwrap(Session.class);           Ⓐ
Blob blob = session.getLobHelper()                     Ⓑ
    .createBlob(imageInputStream, byteLength);
someItem.setImageBlob(blob);
em.persist(someItem);

```

Ⓐ We need the native Hibernate API, so we have to unwrap the `Session` from the `EntityManager`.

Ⓑ Then we need to know the number of bytes we want to read from the stream.

Finally, Hibernate provides fallback serialization for any property type that is `java.io.Serializable`. This mapping converts the value of the property to a byte stream stored in a `VARBINARY` column. Serialization and deserialization occur when the owning entity instance is stored and loaded. Naturally, you should use this strategy with extreme caution because data lives longer than applications. One day, nobody will know what those bytes in the database mean. Serialization is sometimes useful for temporary data, such as user preferences, login session data, and so on.

Hibernate will pick the right type of adapter depending on the Java type of the property. If you don't like the default mapping, read on to override it.

You have seen many adapters and their Hibernate names in the previous sections. Use the name when overriding Hibernate's default type selection and explicitly select a particular adapter:

```
@Entity
public class Item {
    @org.hibernate.annotations.Type(type = "yes_no")
    private boolean verified = false;
}
```

Instead of `BIT`, this `boolean` now maps to a `CHAR` column with values `Y` or `N`.

You can also override an adapter globally in the Hibernate boot configuration with a custom user type, which we'll demonstrate how to write in the next section:

```
metaBuilder.applyBasicType(new MyUserType(), new String[]{"date"});
```

This setting will override the built-in `date` type adapter and delegate value conversion for `java.util.Date` properties to the custom implementation.

We consider this extensible type system to be one of Hibernate's core features and an important aspect that makes it so flexible. Next we'll explore the type system and JPA custom converters in more detail.

6.3.2 Creating custom JPA converters

A new requirement for the online auction system is using multiple currencies, and rolling out this kind of change can be complex. We have to modify the database schema, we may have to migrate existing data from the old to the new schema, and we have to update all applications that access the database. In this section, we'll demonstrate how JPA converters and the extensible Hibernate type system can assist in this process, providing an additional, flexible buffer between the application and the database.

To support multiple currencies, we'll introduce a new class in the `CaveatEmptor` domain model: `MonetaryAmount`, shown in the following listing.

Listing 6.15 Immutable `MonetaryAmount` value-type class

Path: Ch06/mapping-value-types2/src/main/java/com/manning/javapersisten
➡ /ch06/model/MonetaryAmount.java

```
public class MonetaryAmount implements Serializable {                                ④
    private final BigDecimal value;                                                ⑤
    private final Currency currency;                                              ⑥

    public MonetaryAmount(BigDecimal value, Currency currency) {                ⑦
        this.value = value;
        this.currency = currency;
    }

    public BigDecimal getValue() {                                                ⑧
        return value;
    }

    public Currency getCurrency() {                                              ⑨
        return currency;
    }

    @Override
    public boolean equals(Object o) {                                             ⑩
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        MonetaryAmount that = (MonetaryAmount) o;
        return Objects.equals(value, that.value) &&
            Objects.equals(currency, that.currency);
    }

    public int hashCode() {                                                       ⑪
        return Objects.hash(value, currency);
    }

    public String toString() {                                                    ⑫
        return value + " " + currency;
    }

    public static MonetaryAmount fromString(String s) {                          ⑬
        String[] split = s.split(" ");
        return new MonetaryAmount(
            new BigDecimal(split[0]),
            Currency.getInstance(split[1])
        );
    }
}
```

④ This value-typed class should be `java.io.Serializable`: When Hibernate stores entity instance data in the shared second-level cache, it *disassembles* the entity's state. If an entity has a `MonetaryAmount` property, the serialized representation of the property value is stored in the

second-level cache region. When entity data is retrieved from the cache region, the property value is deserialized and reassembled.

- Ⓑ The class defines the value and currency fields, a constructor using both of them, and getters on these fields.
- Ⓒ The class implements the `equals()` and `hashCode()` methods and compares monetary amounts “by value.”
- Ⓓ The class implements the `toString()` method.
- Ⓔ The class implements a static method to create an instance from a `String`.

CONVERTING BASIC PROPERTY VALUES

As is often the case, the database folks can’t implement multiple currencies right away. All they can provide quickly is a column data type change in the database schema.

We’ll add the `buyNowPrice` field to the `Item` class.

```
Path: Ch06/mapping-value-types/src/main/java/com/manning/javapersistenc
➡ /ch06/model/Item.java
```

```
@NotNull
@Convert(converter = MonetaryAmountConverter.class)
@Column(name = "PRICE", length = 63)
private MonetaryAmount buyNowPrice;
```

We’ll store the `BUYNOWPRICE` in the `ITEM` table in a `VARCHAR` column and we’ll append the currency code of the monetary amount to its string value. We will store, for example, the value `11.23 USD` or `99 EUR`.

We’ll convert an instance of `MonetaryAmount` to such a `String` representation when storing data. When loading data, we’ll convert the `String` back into a `MonetaryAmount`. The simplest solution for this is to implement a standardized extension point in JPA, `javax.persistence.AttributeConverter`, in the `MonetaryAmountConverter` class used in the `@Convert` annotation in the preceding snippet. It’s shown in the next listing.

Listing 6.16 Converting between strings and `MonetaryValue`

```
Path: Ch06/mapping-value-types2/src/main/java/com/manning/javapersisten
➡ /ch06/converter/MonetaryAmountConverter.java
```

```
@Converter
```

```

public class MonetaryAmountConverter
    implements AttributeConverter<MonetaryAmount, String> {

    @Override
    public String convertToDatabaseColumn(MonetaryAmount monetaryAmount)
        return monetaryAmount.toString();
    }

    @Override
    public MonetaryAmount convertToEntityAttribute(String s) {
        return MonetaryAmount.fromString(s);
    }
}

```

Ⓐ A converter has to implement the `AttributeConverter` interface; the two arguments are the type of the Java property and the type in the database schema. The Java type is `MonetaryAmount`, and the database type is `String`, which maps, as usual, to an SQL `VARCHAR`. We must annotate the class with `@Converter`.

Ⓑ The `convertToDatabaseColumn` method will convert from the `MonetaryAmount` entity type to the string database column.

Ⓒ The `convertToEntityAttribute` method will convert from the string database column to the `MonetaryAmount` entity type.

To test the functionality of the persistence code, we'll use the Spring Data JPA framework, as demonstrated in the following listing. The source code for this book also contains the testing code alternatives that use JPA and Hibernate.

Listing 6.17 Testing the functionality of the persistence code

Path: Ch06/mapping-value-types2/src/test/java/com/manning/javapersisten
 ➡ /ch06/MappingValuesSpringDataJPATest.java

```

@ExtendWith(SpringExtension.class)
@ContextConfiguration(classes = {SpringDataConfiguration.class})
public class MappingValuesSpringDataJPATest {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private ItemRepository itemRepository;

    @Test
    void storeLoadEntities() {

        City city = new City();
    }
}

```

```

city.setName("Boston");
city.setZipcode("12345");
city.setCountry("USA");
User user = new User();
user.setUsername("username");
user.setHomeAddress(new Address("Flowers Street", city));
userRepository.save(user);

Item item = new Item();
item.setName("Some Item");
item.setMetricWeight(2);
item.setBuyNowPrice(new MonetaryAmount(
    BigDecimal.valueOf(1.1), Currency.getInstance("USD")));
item.setDescription("descriptiondescription");
itemRepository.save(item);

List<User> users = (List<User>) userRepository.findAll();
List<Item> items = (List<Item>)
    itemRepository.findByMetricWeight(2.0);

assertAll(
    () -> assertEquals(1, users.size()),
    () -> assertEquals("username", users.get(0).getUsername()),
    () -> assertEquals("Flowers Street",
        users.get(0).getHomeAddress().getStreet()),
    () -> assertEquals("Boston",
        users.get(0).getHomeAddress().getCity().getName()),
    () -> assertEquals("12345",
        users.get(0).getHomeAddress().getCity().getZipcode()),
    () -> assertEquals("USA",
        users.get(0).getHomeAddress().getCity().getCountry()),
    () -> assertEquals(1, items.size()),
    () -> assertEquals("AUCTION: Some Item",
        items.get(0).getName()),
    () -> assertEquals("1.1 USD",
        items.get(0).getBuyNowPrice().toString()),
    () -> assertEquals("descriptiondescription",
        items.get(0).getDescription()),
    () -> assertEquals(AuctionType.HIGHEST_BID,
        items.get(0).getAuctionType()),
    () -> assertEquals("descriptiond...",
        items.get(0).getShortDescription()),
    () -> assertEquals(2.0, items.get(0).getMetricWeight()),
    () -> assertEquals(LocalDate.now(),
        items.get(0).getCreatedOn()),
    () ->
        assertTrue(ChronoUnit.SECONDS.between(
            LocalDateTime.now(),
            items.get(0).getLastModified()) < 1),
    () -> assertEquals(new BigDecimal("1.00"),
        items.get(0).getInitialPrice())
);

```

```
}  
}
```

- Ⓐ Extend the test using `SpringExtension`. This extension is used to integrate the Spring test context with the JUnit 5 Jupiter test.
- Ⓑ The Spring test context is configured using the beans defined in the `SpringDataConfiguration` class.
- Ⓒ A `UserRepository` bean is injected by Spring through autowiring.
- Ⓓ An `ItemRepository` bean is injected by Spring through autowiring. This is possible as the `com.manning.javapersistence.ch06.repositories` package where `UserRepository` and `ItemRepository` are located was used as the argument of the `@EnableJpaRepositories` annotation on the `SpringDataConfiguration` class. To recall what the `SpringDataConfiguration` class looks like, refer to chapter 2.
- Ⓔ Create and set a city.
- Ⓕ Create and set a user.
- Ⓖ Save it to the repository.
- Ⓗ Create and set an item.
- Ⓘ Save it to the repository.
- Ⓙ Get the list of all users.
- Ⓚ Get the list of items having the metric 2.0.
- Ⓛ Check the size of the list of users.
- Ⓜ Check the name of the first user in the list.
- Ⓝ Check the street address of the first user in the list.
- Ⓞ Check the city of the first user in the list.
- Ⓟ Check the ZIP code of the first user in the list.
- Ⓠ Check the country of the first user in the list.
- Ⓡ Check the size of the list of items.
- Ⓢ Check the name of the first item.
- Ⓣ Check its current buying price.

- ④ Check its description.
- ⑤ Check the auction type.
- ⑥ Check its short description.
- ⑧ Check its metric weight.

⑨ Check the creation date.

⑩ Check the last modification date and time and the initial price of the first item in the list. The last modification date and time is checked against the current date and time to be within 1 second (allowing for the retrieval delay).

Ⓐ Check the initial price of the first item.

Later, when the DBA upgrades the database schema and offers separate columns for the monetary amount and currency, we'll only have to change the application in a few places. We'll drop the `MonetaryAmountConverter` from the project and make `MonetaryAmount` an `@Embeddable`; it will then map automatically to two database columns. It's easy to selectively enable and disable converters, too, if some tables in the schema haven't been upgraded.

The converter we just wrote is for `MonetaryAmount`, a new class in the domain model. Converters aren't limited to custom classes—we can even override Hibernate's built-in type adapters. For example, we could create a custom converter for some or even all `java.util.Date` properties in the domain model.

We can apply converters to properties of entity classes, like `Item#buyNowPrice` in listing 6.17. We can also apply them to properties of embeddable classes.

CONVERTING PROPERTIES OF COMPONENTS

We've been making a case for fine-grained domain models in this chapter. Earlier we isolated the address information of the `User` and mapped the embeddable `Address` class. We'll continue the process and introduce inheritance with an abstract `Zipcode` class, as shown in figure 6.5. The source code that follows can be found in the `mapping-value-types3` folder.

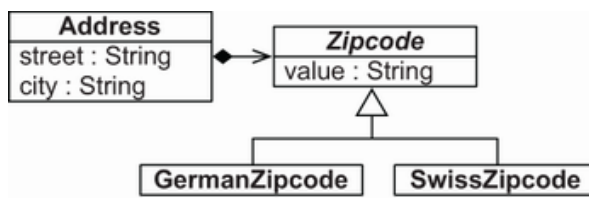


Figure 6.5 The abstract `Zipcode` class has two concrete subclasses.

The `Zipcode` class is trivial, but we have to implement equality by value:

Path: Ch06/mapping-value-types3/src/main/java/com/manning/javapersisten
 ➡ /ch06/model/Zipcode.java

```

public abstract class Zipcode {
    private String value;

    public Zipcode(String value) {
        this.value = value;
    }

    public String getValue() {
        return value;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Zipcode zipcode = (Zipcode) o;
        return Objects.equals(value, zipcode.value);
    }

    @Override
    public int hashCode() {
        return Objects.hash(value);
    }
}

```

We can now encapsulate domain subclasses, the difference between German and Swiss postal codes, and any processing:

Path: Ch06/mapping-value-types3/src/main/java/com/manning/javapersisten
 ➡ /ch06/model/GermanZipcode.java

```

public class GermanZipcode extends Zipcode {
    public GermanZipcode(String value) {
        super(value);
    }
}

```

We haven't implemented any special processing in the subclass. We'll start with the most obvious difference: German ZIP codes are five numbers long, Swiss are four. A custom converter will take care of this.

Listing 6.18 The `ZipcodeConverter` class

```
Path: Ch06/mapping-value-types3/src/main/java/com/manning/javapersisten
➡ /ch06/converter/ZipcodeConverter.java
```

```
@Converter
public class ZipcodeConverter
    implements AttributeConverter<Zipcode, String> {

    @Override                                     ❸
    public String convertToDatabaseColumn(Zipcode attribute) { ❸
        return attribute.getValue();                    ❸
    }                                                    ❸

    @Override                                     ❹
    public Zipcode convertToEntityAttribute(String s) {      ❹
        if (s.length() == 5)                            ❺
            return new GermanZipcode(s);                 ❺
        else if (s.length() == 4)                       ❻
            return new SwissZipcode(s);                  ❻
        throw new IllegalArgumentException(               ❼
            "Unsupported zipcode in database: " + s      ❼
        );                                                ❼
    }                                                    ❼
}
```

❸ Hibernate calls the `convertToDatabaseColumn()` method of this converter when storing a property value; we return a `String` representation. The column in the schema is `VARCHAR`. When loading a value, we examine its length and create either a `GermanZipcode` or `SwissZipcode` instance. This is a custom type discrimination routine; we can pick the Java type of the given value.

❹ Hibernate calls the `convertToEntityAttribute` method of this converter when loading a property from the database.

❺ If the length of the string is 5, a new `GermanZipcode` is created.

❻ If the length of the string is 4, a new `SwissZipcode` is created.

❼ Otherwise, an exception is thrown—the ZIP code in the database is not supported.

Now we'll apply this converter on some `Zipcode` properties, such as the embedded `homeAddress` of a `User`:

```
Path: Ch06/mapping-value-types3/src/main/java/com/manning/javapersisten
➡ /ch06/model/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @Convert(
        converter = ZipcodeConverter.class,
        attributeName = "city.zipcode"
    )
    private Address homeAddress;
    // . . .
}
```

The `attributeName` declares the `zipcode` attribute of the embeddable `Address` class. This setting supports a dot syntax for the attribute path; if `zipcode` isn't a property of the `Address` class but is a property of a nested embeddable `City` class, it is referenced with `city.zipcode`, its nested path.

In JPA 2.2, we can apply several `@Convert` annotations on a single embedded property to convert several attributes of the `Address`. Up to JPA 2.1, we had to group them within a single `@Converts` annotation. We can also apply converters to values of collections, and maps if their values or keys are of basic or embeddable types. For example, we can add the `@Convert` annotation on a persistent `Set<Zipcode>`. We'll demonstrate how to map persistent collections later, with `@ElementCollection`, in chapter 8.

For persistent maps, the `attributeName` option of the `@Convert` annotation has some special syntax:

- On a persistent `Map<Address, String>`, we can apply a converter for the `zipcode` property of each map key with the attribute name `key.zipcode`.
- On a persistent `Map<String, Address>`, we can apply a converter for the `zipcode` property of each map value with the attribute name `value.zipcode`.
- On a persistent `Map<Zipcode, String>`, we can apply a converter for the key of each map entry with the attribute name `key`.
- On a persistent `Map<String, Zipcode>`, we can apply a converter for the value of each map entry by not setting any `attributeName`.

As before, the attribute name can be a dot-separated path if the embeddable classes are nested; we can write `key.city.zipcode` to reference the `zipcode` property of the `City` class in a composition with the `Address` class.

Some limitations of the JPA converters are as follows:

- We can't apply them to identifier or version properties of an entity.
- We shouldn't apply a converter on a property mapped with `@Enumerated` or `@Temporal` because these annotations already declare what kind of conversion has to occur. If we want to apply a custom converter for enums or date/time properties, we shouldn't annotate them with `@Enumerated` or `@Temporal`.

We'll have to change the testing code we wrote a little. We'll replace this line,

```
city.setZipcode("12345");
```

with this one:

```
city.setZipcode(new GermanZipcode("12345"));
```

We'll also replace this line,

```
() -> assertEquals("12345",  
    users.get(0).getHomeAddress().getCity().getZipcode())
```

with this one:

```
() -> assertEquals("12345",  
    users.get(0).getHomeAddress().getCity().getZipcode().getValue())
```

The source code for the book contains these tests using Spring Data JPA, Hibernate, and JPA.

Let's get back to multiple currency support in `CaveatEmptor`. The database administrators have changed the schema again, and we now have to update the application.

6.3.3 Extending Hibernate with UserTypes

Finally, new columns have been added to the database schema to support multiple currencies. The `ITEM` table now has a `BUYNOWPRICE_AMOUNT` and a separate column for the currency of the amount, `BUYNOWPRICE_CURRENCY`. There are also `INITIALPRICE_AMOUNT` and `INITIALPRICE_CURRENCY` columns. We have to map these columns to the `MonetaryAmount` properties of the `Item` class, `buyNowPrice`, and `initialPrice`.

Ideally, we don't want to change the domain model; the properties already use the `MonetaryAmount` class. Unfortunately, the standardized JPA converters don't support the transformation of values from or to multiple columns. Another limitation of JPA converters is integration with the query engine. We can't write the following query: `select i from Item i where i.buyNowPrice.amount > 100`. Thanks to the converter from the previous section, Hibernate knows how to convert a `MonetaryAmount` to and from a string. However, it doesn't know that `MonetaryAmount` has an `amount` attribute, so it can't parse such a query.

A simple solution would be to map `MonetaryAmount` as `@Embeddable`, as you saw earlier in this chapter for the `Address` class (listing 6.13). Each property of `MonetaryAmount` — `amount` and `currency` — maps to its respective database column.

The database admins, however, added a twist to their requirements: because other old applications also access the database, we'll have to convert each amount to a target currency before storing it in the database. For example, `Item#buyNowPrice` should be stored in US dollars, and `Item#initialPrice` should be stored in Euros. (If this example seems far-fetched, we can assure you that you'll see worse in the real world. The evolution of a shared database schema can be costly but is, of course, necessary, because data always lives longer than applications.) Hibernate offers a native converter API: an extension point that allows much more detailed and low-level customization access.

THE EXTENSION POINTS

Hibernate's extension interfaces for its type system can be found in the `org.hibernate.usertype` package. The following interfaces are available:

- `UserType` —You can transform values by interacting with the plain JDBC: `PreparedStatement` (when storing data) and `ResultSet` (when loading data). By implementing this interface, you can also control how Hibernate caches and dirty-checks values.
- `CompositeUserType` —You can tell Hibernate that the `MonetaryAmount` component has two properties: `amount` and `currency`. You can then reference these properties in queries with dot notation, such as `select avg(i.buyNowPrice .amount) from Item i`.

- `ParameterizedType` —This provides settings to the adapter in mappings. We could implement this interface for the `MonetaryAmount` conversion because, in some mappings we'll want to convert the amount to US dollars and in other mappings to Euros. We'll only have to write a single adapter and we can then customize its behavior when mapping a property.
- `DynamicParameterizedType` —This more powerful settings API gives access to dynamic information in the adapter, such as the mapped column and table names. We might as well use this instead of `ParameterizedType`; there is no additional cost or complexity.
- `EnhancedUserType` —This is an optional interface for adapters of identifier properties and discriminators. Unlike JPA converters, a `UserType` in Hibernate can be an adapter for any kind of entity property. Because `MonetaryAmount` won't be the type of an identifier property or discriminator, we won't need it.
- `UserVersionType` —This is an optional interface for adapters of version properties.
- `UserCollectionType` —This rarely-needed interface is used to implement custom collections. We'll have to implement it to persist a non-JDK collection (such as the Google Guava collections: `Multiset`, `Multimap`, `BiMap`, `Table`, etc.) and preserve additional semantics.

The custom type adapter for `MonetaryAmount` will implement several of these interfaces. The source code that follows can be found in the `mapping-value-types4` folder.

IMPLEMENTING THE `USERType`

`MonetaryAmountUserType` is a large class, as you can see in the following listing.

Listing 6.19 The `MonetaryAmountUserType` class

```
Path: Ch06/mapping-value-types4/src/main/java/com/manning/javapersisten
➡ /ch06/converter/MonetaryAmountUserType.java
```

```
public class MonetaryAmountUserType
    implements CompositeUserType, DynamicParameterizedType {

    private Currency convertTo;

    public void setParameterValues(Properties parameters) {
        String convertToParameter = parameters.getProperty("convertTo")
        this.convertTo = Currency.getInstance(
            convertToParameter != null ? convertToParameter : "USD"
        );
    }

    public Class returnedClass() {
```

```

        return MonetaryAmount.class;
    }

    public boolean isMutable() {
        return false;
    }

    public Object deepCopy(Object value) {
        return value;
    }

    public Serializable disassemble(Object value,
                                     SharedSessionContractImplementor session) {
        return value.toString();
    }

    public Object assemble(Serializable cached,
                           SharedSessionContractImplementor session, Object owner) {
        return MonetaryAmount.fromString((String) cached);
    }

    public Object replace(Object original, Object target,
                           SharedSessionContractImplementor session, Object owner) {
        return original;
    }

    public boolean equals(Object x, Object y) {
        return x == y || !(x == null || y == null) && x.equals(y);
    }

    public int hashCode(Object x) {
        return x.hashCode();
    }

    public Object nullSafeGet(ResultSet resultSet,
                              String[] names,
                              SharedSessionContractImplementor session,
                              Object owner) throws SQLException {
        BigDecimal amount = resultSet.getBigDecimal(names[0]);
        if (resultSet.wasNull())
            return null;
        Currency currency =
            Currency.getInstance(resultSet.getString(names[1]));
        return new MonetaryAmount(amount, currency);
    }

    public void nullSafeSet(PreparedStatement statement,
                            Object value, int index,
                            SharedSessionContractImplementor session) throws SQLException {
        if (value == null) {
            statement.setNull(
                index,
                StandardBasicTypes.BIG_DECIMAL.sqlType());
            statement.setNull(

```

```

        index + 1,
        StandardBasicTypes.CURRENCY.sqlType());
    } else {
        MonetaryAmount amount = (MonetaryAmount) value;
        MonetaryAmount dbAmount = convert(amount, convertTo);
        statement.setBigDecimal(index, dbAmount.getValue());
        statement.setString(index + 1, convertTo.getCurrencyCode())
    }
}

public MonetaryAmount convert(MonetaryAmount amount,
                              Currency toCurrency) {
    return new MonetaryAmount(
        amount.getValue().multiply(new BigDecimal(2)),
        toCurrency
    );
}

public String[] getPropertyNames() {
    return new String[]{"value", "currency"};
}

public Type[] getPropertyTypes() {
    return new Type[]{
        StandardBasicTypes.BIG_DECIMAL,
        StandardBasicTypes.CURRENCY
    };
}

public Object getPropertyValue(Object component,
                              int property) {
    MonetaryAmount monetaryAmount = (MonetaryAmount) component;
    if (property == 0)
        return monetaryAmount.getValue();
    else
        return monetaryAmount.getCurrency();
}

public void setPropertyValue(Object component,
                             int property,
                             Object value) {
    throw new UnsupportedOperationException(
        "MonetaryAmount is immutable"
    );
}
}

```

Ⓐ The interfaces we implement are CompositeUserType and DynamicParameterizedType.

Ⓑ The target currency.

- Ⓒ The `setParameterValues` method is inherited from the `DynamicParameterizedType` interface.
- Ⓓ Use the `convertTo` parameter to determine the target currency when saving a value into the database.
- Ⓔ If the parameter hasn't been set, default to US dollars.
- Ⓕ The `returnedClass` method adapts the given class, in this case, `MonetaryAmount`. This method and the ones to follow are inherited from the `CompositeUserType` interface.
- Ⓖ Hibernate can enable some optimizations if it knows that `MonetaryAmount` is immutable.
- Ⓗ If Hibernate has to make a copy of the value, it calls this `deepCopy` method. For simple immutable classes like `MonetaryAmount`, we can return the given instance.
- Ⓘ Hibernate calls the `disassemble` method when it stores a value in the global shared second-level cache. We need to return a `Serializable` representation. For `MonetaryAmount`, a `String` representation is an easy solution. Or, because `MonetaryAmount` is `Serializable`, we could return it directly.
- ⓵ Hibernate calls the `assemble` method when it reads the serialized representation from the global shared second-level cache. We create a `MonetaryAmount` instance from the `String` representation. Or, if we stored a serialized `MonetaryAmount`, we could return it directly.
- Ⓚ The `replace` method is called during `EntityManager#merge()` operations. We need to return a copy of the original. Or, if the value type is immutable, like `MonetaryAmount`, we can return the original.
- Ⓛ Hibernate uses value equality to determine whether the value was changed and the database needs to be updated. We rely on the equality and hash code routines we have already written on the `MonetaryAmount` class.
- Ⓜ The `nullSafeGet` method is called to read the `ResultSet` when a `MonetaryAmount` value has to be retrieved from the database.
- Ⓝ Take the `amount` and `currency` values as given in the query result.
- Ⓞ Create a new instance of `MonetaryAmount`.
- Ⓟ The `nullSafeSet` method is called when a `MonetaryAmount` value has to be stored in the database.

Ⓚ If `MonetaryAmount` was `null`, we call `setNull()` to prepare the statement.

Ⓜ Otherwise, we convert the value to the target currency.

Ⓢ We then set the `amount` and `currency` on the provided `PreparedStatement()`.

Ⓣ We can implement whatever currency conversion routine we need.

Ⓤ For the sake of this example, we double the value so we can easily test whether the conversion was successful. We'd have to replace this code with a real currency converter in a real application. This `convert` method is not a method of the Hibernate `UserType` API.

Ⓥ The remaining methods inherited from `CompositeUserType` provide the details of the `MonetaryAmount` properties, so Hibernate can integrate the class with the query engine. The `getPropertyNames` method will return a `String` array with two elements, `value` and `currency` — the names of the properties of the `MonetaryAmount` class.

Ⓦ The `getPropertyTypes` method will return a `Type` array with two elements, `BIG_DECIMAL` and `CURRENCY` — the types of the properties of the `MonetaryAmount` class.

Ⓧ The `getPropertyValue` method will return either the `value` field or the `currency` field of the `MonetaryAmount` object, depending on the property index.

Ⓨ The `setPropertyValue` method will not allow any field of the `MonetaryAmount` object to be set, as this one is immutable.

The `MonetaryAmountUserType` class is now complete, and we can already use it in mappings with its fully qualified class name in `@org.hibernate.annotations.Type`, as demonstrated in the “Selecting a type adapter” section (in section 6.3.1). This annotation also supports parameters, so we can set the `convertTo` argument to the target currency.

However, we recommend creating *type definitions*, bundling the adapter with some parameters.

USING TYPE DEFINITIONS

We need an adapter that converts to US dollars, and another that converts to Euros. If we declare these parameters once as a *type definition*, we won't have to repeat them in property mappings. A good location for type definitions is package metadata, in a `package-info.java` file:

Path: Ch06/mapping-value-types4/src/main/java/com/manning/javapersisten
➡ /ch06/converter/package-info.java

```
@org.hibernate.annotations.TypeDefs({
    @org.hibernate.annotations.TypeDef(
        name = "monetary_amount_usd",
        typeClass = MonetaryAmountUserType.class,
        parameters = {@Parameter(name = "convertTo", value = "USD")}
    ),
    @org.hibernate.annotations.TypeDef(
        name = "monetary_amount_eur",
        typeClass = MonetaryAmountUserType.class,
        parameters = {@Parameter(name = "convertTo", value = "EUR")}
    )
})
package com.manning.javapersistence.ch06.converter;
import org.hibernate.annotations.Parameter;
```

We are now ready to use the adapters in mappings, using the names
monetary_ amount_usd and monetary_ amount_eur.

We can map the buyNowPrice and initialPrice of Item:

Path: Ch06/mapping-value-types4/src/main/java/com/manning/javapersisten
➡ /ch06/model/Item.java

```
@Entity
public class Item {
    @NotNull
    @org.hibernate.annotations.Type(
        type = "monetary_amount_usd"
    )
    @org.hibernate.annotations.Columns(columns = {
        @Column(name = "BUYNOWPRICE_AMOUNT"),
        @Column(name = "BUYNOWPRICE_CURRENCY", length = 3)
    })
    private MonetaryAmount buyNowPrice;

    @NotNull
    @org.hibernate.annotations.Type(
        type = "monetary_amount_eur"
    )
    @org.hibernate.annotations.Columns(columns = {
        @Column(name = "INITIALPRICE_AMOUNT"),
        @Column(name = "INITIALPRICE_CURRENCY", length = 3)
    })
    private MonetaryAmount initialPrice;
    // . . .
}
```


If `UserType` transforms values for only a single column, we don't need an `@Column` annotation. `MonetaryAmountUserType`, however, accesses two columns, so we need to explicitly declare two columns in the property mapping. Because JPA doesn't support multiple `@Column` annotations on a single property, we'll have to group them with the proprietary `@org.hibernate.annotations.Columns` annotation. Note that the order of the annotations is now important! Recheck the code for `MonetaryAmountUserType`; many operations rely on indexed access of arrays. The order when accessing `PreparedStatement` or `ResultSet` is the same as that for the declared columns in the mapping. Also, note that the number of columns isn't relevant for the choice of `UserType` versus `CompositeUserType`—only the desire to expose value type properties for queries.

We'll have to change the testing code we wrote. We'll add this line to set the `Item`:

```
item.setBuyNowPrice(new MonetaryAmount(BigDecimal.valueOf(1.1),
                                     Currency.getInstance("USD")));
```

We'll replace this line,

```
() -> assertEquals("1.1 USD",
                   items.get(0).getBuyNowPrice().toString())
```

with this one:

```
() -> assertEquals("2.20 USD",
                   items.get(0).getBuyNowPrice().toString())
```

We'll also replace this line,

```
() -> assertEquals(new BigDecimal("1.00"),
                   items.get(0).getInitialPrice())
```

with this one:

```
() -> assertEquals("2.00 EUR",
                   items.get(0).getInitialPrice().toString())
```

This is because the `convert` method from the `MonetaryAmountUserType` class doubles the value of the amount (shown in listing 6.19 at ①). The source code for this book contains the tests using Spring Data JPA, Hibernate, and JPA.

With `MonetaryAmountUserType`, we've extended the buffer between the Java domain model and the SQL database schema. Both representations are now more robust with respect to changes, and we can handle even rather eccentric requirements without modifying the essence of the domain model classes.

Summary

- You can map the basic and embedded properties of an entity class.
- You can override basic mappings, change the name of a mapped column, use derived, default, temporal, and enumeration properties, and test them.
- You can implement embeddable component classes and create fine-grained domain models.
- You can map the properties of several Java classes in a composition, such as `Address` and `City`, to one entity table.
- Any JPA provider supports a minimum set of Java-to-SQL type conversions, as well as some additional adapters.
- You can write a custom type converter, as we did for the `MonetaryAmount` class, with the standard JPA extension interfaces. You can also write a low-level adapter, as we did with the native Hibernate `UserType` API.